

Homework 4: Turing Machines

Due: 03/17/2026

This assignment is due on **11:59 PM, March 17, 2026**. You may turn it in up to 48 hours late, but assignments turned in by the deadline receive 3% extra credit.

You should submit a typeset or *neatly* written pdf on Canvas. The grading TA should not have to struggle to read what you've written; if your handwriting is hard to decipher, you will be asked to typeset your future assignments.

You may collaborate with other students, but any written work should be your own.

Important Note: Please be aware of alternative valid ways of denoting transitions in a TM. Here are a few examples from the Sipser Textbook:

- $0 \rightarrow R$ is shorthand for $0 \rightarrow 0, R$
- $0, 1 \rightarrow R$ is shorthand for $0 \rightarrow 0, R$ and $1 \rightarrow 1, R$

1. **Explicit Construction** (4 points - *Section B only*)

Give an explicit construction (state diagram) of a Turing Machine that, when started with a positive integer x in binary on the tape, halts with $x + 1$ in binary on the tape. Assume the tape is infinite in both directions and the tape head starts at the most significant digit of x .

Answer:

Now similarly, give an explicit construction (state diagram) of a Turing Machine that, when started with a positive integer x in binary on the tape, halts with $x - 1$ in binary on the tape. Assume the tape is infinite in both directions and the tape head starts at the least significant digit of x .

Answer:

2. **Slightly Less Explicit Construction** (4 points - *Section B only*)

For this problem, you will describe a TM M that takes as input a DFA D and an input x , and returns whether D accepts x . For simplicity, assume that the input alphabet for D is $\{0, 1\}$. (Note: D is part of the *input*. You are not building a different Turing machine for each possible DFA, you are building a single Turing machine that can simulate every DFA.)

- (2 points) Let $\Sigma_M = \{0, 1, \#\}$ be the input alphabet for M . An input to M is a ordered pair (D, x) . Describe an appropriate encoding scheme for the input to M .

Answer:

- (2 points) Describe, at a higher level than problem 1, how M runs. Your description does not need to explain how to check if two strings are equal but should otherwise describe the path that the tape head will follow as M runs.

Answer:

3. Conversion (4 points - Both Sections)

Let a kL - mR Turing Machine be a Turing Machine whose transition function requires the head to move **exactly** k squares to the left (kL) or **exactly** m squares to the right (mR) on each step, where $k, m \in \mathbb{Z}_{>0}$.

Show that kL - mR Turing Machines are exactly as powerful as standard Turing Machines by proving equivalence in both directions:

- Given any kL - mR TM, construct an equivalent standard TM.
- Given any standard TM, construct an equivalent kL - mR TM.

Your proof must be precise enough to show how to simulate one model with the other. Note that this proof has two directions, and you must prove both of them.

Answer: $\Rightarrow$ We show that given a kl - mR TM, we can construct an equivalent TM. First consider the 7 tuple associated with the kl - mR TM, i.e. $(Q, \Sigma, \Gamma, \delta', q_0, q_{acc}, q_{rej})$. Now we know that δ' is such that we move exactly k squares to the left or m squares to the right. So we have $\delta'_{km}(p, \alpha) = (q, \beta, kL)$ meaning we move k to the left or $\delta'_{km}(p, \alpha) = (q, \beta, mR)$ meaning we move m to the right. Now simply define our new TM with the same 7 tuple but with a new transition function δ and set of states Q' , so we have $(Q', \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$. However, we define δ now such that for any $\delta'_{km}(p, \alpha) = (q, \beta, kL)$ we define states $p_1, \dots, p_{k-1}$ such that $\delta(p, \alpha) = (p_1, \beta, L)$ and $\delta(p_i, x) = (p_{i+1}, x, L)$ for any $1 \leq i \leq k-2$ and lastly $\delta(p_{k-1}, x) = (q, x, L)$ where x is any tape symbol in Γ . Now similar for the right side we also convert a given $\delta'_{km}(p, \alpha) = (q, \beta, mR)$ to an equivalent set of transitions using δ by defining $p_1, \dots, p_{m-1}$ where we define $\delta(p, \alpha) = (p_1, \beta, R)$ and for each $1 \leq i \leq m-2$ we have, $\delta(p_i, x) = (p_{i+1}, x, R)$ and $\delta(p_{m-1}, x) = (q, x, R)$ where x is some symbol in Γ . So note that Q' is defined as Q along with the additional states we just defined here.

$\Leftarrow$ Now given a standard TM we need an equivalent kL - mR TM. Essentially we need to simulate a standard TM using the kL - mR TM. We need the ability to go right and left with kL 's and mR 's. Now note that given arbitrary k, m the minimal possible move we can make would be $\gcd(k, m)$, so if we want a single step, we can only do it if $\gcd(k, m) = 1$. So assume this is the case.

Now note that if $\gcd(k, m) = 1$ we can find $a, b \in \mathbb{Z}$ such that $ak + bm = 1$ such that a and b have opposite signs. This is possible because we know that $k, m \in \mathbb{Z}^+$ and we can construct infinite solutions for a, b by considering $a = a_0 \pm tm$ and $b = b_0 \mp tk$, where a_0, b_0 are some initial solution. Now note that at some point (WLOG consider

we're increasing the value of a) ak is positive greater than 1 (because we can keep increasing the value and we're in $\mathbb{Z}$). But if $ak > 1$ then $1 - ak = bm < 0$ but $m > 0$ so $b < 0$. So we have a positive and b negative. Similarly we can find a negative and b positive.

Now note that we need to simulate a R turn and an L turn using the kL and mR. First for a R turn we essentially want $ak + bm = 1$ where $a < 0$ (as that corresponds to k going left) and b positive. We know we can find this as we showed above. Hence, we can simulate a right turn say $\delta(p, \alpha) = (q, \beta, R)$ by doing $\delta'(p, \alpha) = (q_1, \beta, mR)$ and $\delta'(q_i, X) = (q_{i+1}, X, mR)$ for i from 1 to $b-2$ and we add $\delta(q_{b-1}, X) = (r_1, X, kL)$ and $\delta(r_i, X) = (r_{i+1}, X, kL)$ for i from 1 to $|a|-2$ and we lastly add $\delta(r_{|a|-1}, X) = (q, X, kL)$ (Note here that X means any symbol). Now for a L turn note that we need $ak + bm = -1$ we can get this by choosing a, b such that $ak + bm = 1$ but where a is positive and b is negative and note that if we multiply the expression by -1 then we have $(-a)k + (-b)m = -1$ but $-a$ is now negative and $-b$ is now positive (which corresponds to moving left for k and moving right for m). We do a similar thing as the case above for the right turn to now get the left turn.

Now technically for the case where $\gcd(k, m) = d > 1$, we can show an equivalence of this with a dR, dL standard Turing machine. This Turing machine can be shown to be the same as a standard Turing machine if we modify the input so that we allow it to skip d symbol's. i.e we modify the input from $x_1x_2x_3 \dots x_n$ to

$$\underbrace{x_1x_1 \dots x_1}_{d \text{ times}} \underbrace{x_2x_2 \dots x_2}_{d \text{ times}} \dots \underbrace{x_nx_n \dots x_n}_{d \text{ times}}$$

Now a single dR turn is equivalent to a single R turn in the standard TM.

4. Unrestricted grammars (4 points - Both Sections)

An unrestricted grammar is similar to a context-free grammar, except that its rules are of the form $u \rightarrow v$, where u and v are *both* strings containing any number of terminals and non-terminals.

- (1 point) Here is an unrestricted grammar:

$$\begin{aligned} S &\rightarrow \varepsilon \\ S &\rightarrow aASCc \\ Aa &\rightarrow aA \\ cC &\rightarrow Cc \\ bC &\rightarrow Cb \\ AC &\rightarrow b \end{aligned}$$

What language does this grammar generate?

Answer: The grammar generates $a^m b^m c^m$ for $m \geq 0$ because repeated applying the second production for S gives us $a^m A^m C^m c^m$ where $S \rightarrow \varepsilon$ is applied at the end. Then the middle two become b^m as we apply the other productions.

- (1 point) Let L be a language generated by an unrestricted grammar G . Show that L is Turing-recognizable. (Do NOT give an example for a specific language. Your answer should be an explanation for how to build an equivalent Turing Machine for any unrestricted grammar.)

Answer: Consider a 2 tape Turing machine. So in the first tape we have our input string and in the second tape we have our start symbol S . Now given our productions we non-deterministically explore all possible productions, i.e we use the productions for any occurrence of the LHS in our string and expand it out. Now if the string consists only of terminals, then we can compare the string to the bottom tape and accept if it is the same. If w is in our language then there exists some set of productions that will bring it, because we're looking through all possible productions, we'll reach it at some point (Although we can't guarantee when) this means it's possible also or us to loop continuously not reaching an accept or reject state. So because for any $w \in L$ our TM accepts it is recognizable.

- (2 points) Show that, for any Turing-recognizable L , there is an unrestricted grammar that generates L . (Hint: We showed in class how to represent the configuration of a Turing machine as a string, i.e. $001q_700$. At each step, the configuration of the machine changes. Think about how the configuration string changes, and represent this change with a grammar rule.)

Answer: Now consider a Turing recognizable language L . We have a Turing machine associated with this language. Now note that our δ is the transition function that moves a given configuration state to the other and a string is accepted when we reach q_{acc} . First we need to represent the initial configuration with our TM, this means we need to represent $q_0 w$ where w is our input. So we need to be able to generate some arbitrary input. To do this consider we have $S \rightarrow q_0 W$ and $W \rightarrow aW \mid \varepsilon$ for any input symbol $a \in \Sigma$. So this can generate $S \rightarrow q_0 w$ where w is a input string.

Now we need a set of production rules that encode the transitions from δ and the accepting state as a terminal. Now consider a delta transition $\delta(p, \alpha) = (q, \beta, R/L)$ that moves a configuration of $xxp\alpha xxx$ to $xx\beta q xxx$. We can represent this as a grammar rule $p\alpha \rightarrow \beta q$ and for left similarly we have $xp\alpha \rightarrow qx\beta$ where x is any tape symbol in Γ . And if we add this for each transition in our TM we replicate the functionality of our Turing machine. Now note that we need rules for ending as well. For ending we add rules to remove the non-input strings. So we have $q_{acc} \rightarrow \varepsilon$ as well as $aq_{acc} \rightarrow q_{acc}$ and $q_{acc}a \rightarrow q_{acc}$ for any tape symbol $a \in \Gamma$ and note that because we only do the cleanup for a in gamma, we only remove the tape symbols, which means we're left with only the real input symbols.

5. More Conversion (4 points - Section X only)

We define two non-standard models of Turing Machines:

- A **Reset-Turing Machine** (Reset-TM) has a right-infinite tape. Its head cannot move left by one square. Instead, it may either move right by one square (R) or *reset* directly to the left endmarker $\vdash$.
- A **Skip-Turing Machine** (Skip-TM) is a single-tape machine whose head cannot move one square at a time. Instead, each step must be either move two squares left ($2L$) or two squares right ($2R$).

You need to prove that Reset-TMs and Skip-TMs are equivalent in computational power. That is, for every Reset-TM M there exists a Skip-TM M' with the same language, and vice versa. Note that this proof has two directions, and you must prove both of them.

Hints:

- You may simulate one model by going through a standard TM as an intermediate step:

$$\text{Reset-TM} \iff \text{standard TM} \iff \text{Skip-TM}.$$

- If you aim for a more direct construction, focus on how to simulate **reset** using only $2L/2R$, and how to simulate a single L or R using **reset** + R .

Answer:

We will show this by showing an equivalence of the Reset-TM and the Skip-TM with the standard TM.

1. First we show that Reset-TM $\iff$ standard-TM.

$\implies$ Assume we have a standard TM, we will now simulate it within a reset TM. Note that for any right move in the standard TM we can simply take a right move in the reset TM as well. Now consider a left move in the standard TM. i.e say we have $\delta(q, \alpha) = (p, \beta, L)$. Now note that we can only take an R or *Reset* in our reset TM. So to simulate a left turn we need to reset and take rights until we're 1 before our previous configuration. Our construction will attempt to do the following, given we're in symbol α we will first mark α to a special α' which represent the previous position. Now we'll reset to the start to a state c_1 . We define new states c_i which represent the fact that we'll be assuming the left is at the i 'th position to the left (the transversal to the i 'th position will be done by introducing states $c_{i1}, \dots, c_{ii}$, we can ensure it only goes i to the right because we don't have states $c_{i(i+1)}$). Once we guess that it's in the i 'th position, we mark the current position say α_i to α_i^m and then move right. Now we check if the right symbol is the marked symbol ie. α' that we marked earlier. If it isn't we reset to state c_{i+1} and repeat. (Note that when we traverse if we ever encounter a α^m we reset it to α). Now this checks each position if it's the left marks it moves right if right is not marked it resets and fixes it's previous

markings. Now consider the case where the right is the correct previous i.e the right is marked. Now we move to a special state say c_{im} which represents the confirmation of the index of the left. When resetting we unmark α' to β . Then we do our counting again by using states $c_{im1} \dots c_{imi}$ and reach the state which is now marked with m . We can unmark this back to what it was and reset again using $c_{im'}$ which represents the fact that everything is fixed and all we need to do is go to the i 'th position and change the state to p as required. (Note that for each state p we're moving to the left to we need $c_{ip}, c_{ipm}, c_{ipm'}$ so that we know what to change the left state by, but for simplicity I don't mention this above, also note we're extend our alphabet so that for a tape symbol a we also have a' which represents the right symbol essentially and symbol a^m that represents the left symbol) .

$\Leftarrow$ Now given a reset TM we use a standard TM to simulate it. First in our standard TM we add an extra symbol in the start that represents the start of our input, our standard TM is then initialized with q_0 right after that (so it's equivalent to how a standard TM starts). Now for every right turn we can simply do the right turn in the standard TM as well. Now to simulate the reset we need to use the left turns to move to the beginning, so for any reset i.e $\delta(q, \alpha) = (p, \beta, Reset)$ what we'll do is add states $\delta(q, \alpha) = (q', \beta, L)$ and then we continue moving left $\delta(q', X) = (q', X, L)$ where X is any symbol except the start symbol that we added. So this means that the left turns will end when we reach the start symbol because there are no transitions defined to move left from the start symbol. Once we get the start symbol we just move right one bit to represent the restart i.e we have $\delta(q', S) = (p, S, R)$ where S is the start symbol and p is the state we wanted to end up in the reset TM.

2. Now we show equivalence between the skip TM and standard TM.

$\Rightarrow$ Assume we have a skip TM, we'll simulate a standard TM in it. First given an input alphabet $x_1 \dots x_n$ we'll encode this as $x_1 x_1 \dots x_n x_n$ in our skip TM. Now for any right in the standard TM we take our 2R in the skip TM and similarly for any left we take 2L. So we have $\delta(p, \alpha) = (q, \beta, R/L)$ from the standard to get $\delta'(p, \alpha) = (q, \beta, 2R/2L)$.

$\Leftarrow$ Assume we have a standard TM and we want to simulate a skip TM. Simply copy the input the same way and now for any 2R in the skip TM, we just do a right 2 times or left 2 times. So if we have $\delta(p, \alpha) = (q, \beta, 2R)$ then we can do $\delta'(p, \alpha) = (p_{1q}, \beta, R)$ and $\delta'(p_{1q}, X) = (q, X, R)$ where X is any tape symbol.

So we just showed the equivalence of both to the standard TM which means they both are equivalent to each other.

6. Fun with Enumerators (4 points - Section X only)

- (2 points) Fix an ordering R of the strings of Σ^* . Show that L is decidable if and only if there is an enumerator for L that prints the strings of L in order

according to R .

Answer: $\Rightarrow$ First assume that L is decidable. Now this means for a given input string if it's in L our TM M will accept, else we reject. Now we need to find an enumerator that prints the strings of L in any order according to R . Let the order be as $s_1, s_2, s_3, \dots$. Now we define our enumerator as follows, we enumerate M on each s_i for $i = 1, 2, 3, \dots$ and for each s_i we check if M accepts it or rejects it. If M accepts it, we print it out if M rejects it we move on. So now because we're going sequentially i.e $s_1, s_2, \dots$ respecting the same order, our enumerator E will also respect the same order R . (Note this can only be done if M is decidable because otherwise it's possible that we get stuck on some s_i without reaching a accept or reject state)

$\Leftarrow$ Now assuming we have an enumerator that respects any order R imposed on Σ^* of L . Now we need to show that L is Turing decidable. We construct M as follows, first fix an order for R . Now note that for a fixed order we choose for a given input string w , there is a finite position in our order that it exists in. So on our machine M , we run the enumerator E . For each string E prints x we compare x with w . If it is the same we accept, now note that in a fixed order because there is a finite index for w there are only finitely many strings before it. So if we pass w and we didn't print it out it implies it is not in our language (we have a notion of checking whether we passed w because we R is some algorithmic order and hence we can compute whether a string is before or after to another string according to the order R). In this case we can just reject the string. Hence for any $w \in L$ we accept else we reject.

- (2 points) Suppose $L_1 = \{\langle M_1 \rangle, \langle M_2 \rangle, \dots\}$ is a Turing-recognizable language whose strings are Turing machine descriptions. Show that there is a decidable language L_2 such that for each machine described by a string in L_1 , there is an equivalent machine described by a string in L_2 , and vice-versa. (Hint: start with an enumerator for L_1 , and use the result above.)

Answer: We have L_1 is a Turing recognizable language. If L_1 is TM-Recognizable, there exists an enumerator that enumerates it. So we have an enumerator that prints out the our TM descriptions in some order $M_1, M_2, \dots$. Now we need an enumerator for the language L_2 . So what we'll do is we define M'_i as follows, for each input for our machine, we run the enumerator until we get M_i now for an input w we simulate $M_i(w)$. So essentially they share the same language and hence are equivalent. Now consider the language $L_2 = \{M'_1, M'_2, \dots\}$. Now we need to show that there is an enumerator that generates this language in a fixed order R . Now assume a fixed order R , we make a new enumerator as follows, for a given w we first print out the first TM, simulate it, then we check if it's the first string by R , if not we print the next string and continue the same, if it's the first string in R then we print it out, and then we repeat the process. (note that when checking if it's the next string in R , we look at every string that we computed till now). SO this means that we have an enumerator for any order

R for our language. This means that our language is decidable because of the above theorem.